

Rapport de Projet

Simulation 2D de Cellules

Application Java / JavaFX

Projet de Génie Logiciel — ING1-GI

Youssef Missaoui • Rihem Sahli • Asaad Nasri • Jude Owona Assoumou • Laurent Minkoulou

Dépôt GitHub : <https://github.com/judeown/Projet-G-nie>

Année universitaire 2025–2026

Sommaire

Sommaire	1
1. Présentation de l'équipe	1
2. Présentation du sujet	1
3. Objectif de l'application	1
4. Organisation du groupe	1
5. Répartition des tâches	1
6. Problèmes rencontrés	1
7. Solutions apportées	1
8. Résultats obtenus	1
9. Limites du projet	1
10. Améliorations possibles	1
11. Diagramme UML	1
12. Cas d'utilisation	1
Conclusion	1

1. Présentation de l'équipe

Le projet a été réalisé par le groupe d'étudiants ING1 GI5-C dans le cadre du cours de Projet de Génie Logiciel. L'équipe s'est organisée en cinq parties principales, chacune correspondant à un bloc technique du projet. Cette répartition permettait à chaque membre d'avancer sur une zone précise tout en préparant l'intégration finale avec les autres modules.

Membre	Rôle principal dans le projet
Laurent	Partie 1 — Cellules / Agents : création des agents, de leurs états et de leurs comportements de base.
Rihem	Partie 2 — Grille + Moteur de simulation : gestion de la grille, des positions et de l'évolution de la simulation.
Asaad	Partie 3 — Interface JavaFX : création de la fenêtre graphique, affichage de la grille et des contrôles.
Jude	Partie 4 — Interactions utilisateur : gestion des actions souris/clavier, ajout, suppression, pinceau et gomme.
Youssef	Partie 5 — Statistiques + Sauvegarde + CLI : calcul des statistiques, sauvegarde/chargement et version console.

Cette organisation montre une volonté de séparer les responsabilités : chaque membre peut travailler sur une partie relativement indépendante, tout en devant ensuite intégrer son travail dans une application unique. Le projet met donc en pratique plusieurs compétences importantes : programmation orientée objet, conception modulaire, interface graphique, manipulation de fichiers et collaboration avec Git/GitHub.

2. Présentation du sujet

Le sujet consiste à développer une application capable de simuler l'évolution de cellules ou d'agents sur une grille bidimensionnelle. Chaque case de la grille peut contenir une cellule possédant un état de santé, un âge, une énergie et une probabilité de déplacement. L'ensemble forme une simulation dynamique dans laquelle les cellules peuvent interagir avec leur environnement et avec les cellules voisines.

L'application reprend une logique proche des automates cellulaires et des simulations multi-agents. Le comportement global n'est pas programmé directement ; il apparaît progressivement à partir de règles simples appliquées à chaque cellule : propagation de l'infection, récupération, mort, déplacement, vieillissement, perte d'énergie et division.

Le projet dépasse donc le simple affichage d'une grille. Il combine un modèle de données, un moteur de simulation probabiliste, une interface graphique interactive, un module de statistiques en temps réel et un système de sauvegarde/chargement.

3. Objectif de l'application

L'objectif principal de l'application est de permettre à un utilisateur de créer, lancer, observer et modifier une simulation de cellules sur une grille 2D. Voici les fonctionnalités principales :

- Afficher une grille 2D configurable de 40 colonnes par 30 lignes dans la version JavaFX.
- Permettre l'ajout, la suppression et le placement rapide de cellules avec plusieurs outils d'interaction.
- Simuler la propagation d'une infection entre cellules voisines selon une probabilité dépendant de l'âge.
- Gérer la récupération, la mort, le déplacement, le vieillissement, l'énergie et la division des cellules.
- Afficher des statistiques en temps réel : nombre d'agents par état, pourcentages, âge moyen, énergie moyenne.
- Proposer une sauvegarde et un chargement d'une simulation au format binaire.
- Fournir une version console pour vérifier le fonctionnement du modèle sans interface graphique.

4. Organisation du groupe

L'organisation du groupe a été pensée en fonction des dépendances techniques du projet. Le travail a été découpé en cinq parties permettant de commencer par les éléments les plus indépendants avant d'intégrer progressivement les parties dépendantes.

Partie	Nom	Dépendance
1. Cellules / Agents	Laurent	Presque indépendante : cette partie a seulement besoin de représenter la position des agents avec x et y.
2. Grille + Moteur de simulation	Rihem	Dépend surtout des cellules : le moteur manipule des objets Agent et applique les règles d'évolution.
3. Interface JavaFX	Asaad	Dépend du moteur : l'interface affiche l'état de la grille et pilote la simulation. Elle peut toutefois commencer avec une grille de test.
4. Interactions utilisateur	Jude	Dépend de JavaFX et de Grid : les clics et outils doivent modifier correctement la grille affichée.
5. Statistiques + Sauvegarde + CLI	Youssef	Dépend surtout du moteur et de la grille : les statistiques lisent les agents, et la sauvegarde conserve l'état de la simulation.

Cette organisation évite que tout le monde travaille sur les mêmes fichiers au même moment. La partie Cellules / Agents devait être définie rapidement, car elle sert de base au moteur. La grille et le moteur alimentent ensuite l'interface JavaFX. Les interactions utilisateur dépendent de l'interface et de la grille, tandis que les statistiques, la sauvegarde et la CLI peuvent être préparées avec des données de test.

5. Répartition des tâches

La répartition des tâches a suivi le découpage en cinq parties. Chaque membre avait une responsabilité principale, mais les modules devaient ensuite être reliés entre eux pour former une application complète.

Partie	Responsable	Travail réalisé
--------	-------------	-----------------

Partie 1 — Cellules / Agents	Laurent	Création des « personnes » de la simulation avec Agent et HealthState. Gestion des états HEALTHY, INFECTED, RECOVERED et DEAD. Attributs principaux : x, y, age, energy, state, infectionTime et moveProbability. Méthodes : infect(), recover(), die(), isAlive(), updateAge() et canMove().
Partie 2 — Grille + Moteur	Rihem	Création du cœur du projet avec Grid, SimulationEngine et Position. Fonctions principales : start(), pause(), step(), update(), getNeighbors(), moveAgent(), addAgent() et removeAgent().
Partie 3 — Interface JavaFX	Asaad	Création de la fenêtre graphique : affichage de la grille, boutons, couleurs des agents, mise à jour visuelle, Canvas, slider de vitesse et menus de choix des cellules.
Partie 4 — Interactions utilisateur	Jude	Gestion des interactions souris/clavier : cliquer pour ajouter un agent, clic droit pour supprimer, pinceau, gomme, modification des paramètres et choix du type d'agent.
Partie 5 — Stats + Sauvegarde + CLI	Youssef	Calcul des statistiques : nombre de cellules saines, infectées, guéries et mortes, âge moyen et énergie moyenne. Gestion de saveSimulation() et loadSimulation(). Création de la version CLI.

6. Problèmes rencontrés

Plusieurs difficultés sont apparues naturellement dans ce type de projet, notamment parce que les cinq parties ne possèdent pas toutes le même niveau de dépendance.

Problème	Conséquence sur le projet
Gestion des coordonnées	Cohérence entre coordonnées de la grille, positions des agents et coordonnées pixel de la souris.
Intégration JavaFX	Le SDK JavaFX et les chemins de compilation dépendent de la machine de chaque membre.
Simulation probabiliste	Les résultats varient à chaque lancement (Math.random()), rendant les tests plus difficiles.
Synchronisation interface/moteur	Les boutons Play, Pause, Step, Reset, etc. doivent rester cohérents avec l'état réel du moteur.
Statistiques et classes de test	Coexistence d'une version de test avec des classes mock et de la version principale.
Sauvegarde des états	La sauvegarde doit conserver agents, positions, états, âge, énergie, topologie et pas de simulation.

Un autre problème important est la complexité progressive du projet. Au départ, une grille et quelques cellules suffisent. Mais dès que l'on ajoute les boutons, les sliders, les modes d'interaction, les statistiques et les fichiers de sauvegarde, il devient nécessaire de bien séparer les responsabilités.

7. Solutions apportées

Les solutions apportées reposent principalement sur la modularisation du projet et sur le découpage clair des responsabilités. Chaque difficulté a été associée à une classe ou à un groupe de classes spécifique :

- La classe Grid centralise le stockage des agents et la recherche de voisins libres ou occupés.
- La classe Position fournit des méthodes statiques pour calculer les voisins, y compris dans le cas d'une grille toroïdale.
- La classe SimulationEngine isole les règles d'évolution afin que l'interface graphique n'ait pas à connaître le détail de l'infection ou de la division.
- La classe InteractionController convertit les actions de souris en opérations concrètes sur la grille avec quatre outils : ADD, REMOVE, BRUSH et ZONE.
- La classe StatisticsManager calcule les indicateurs sans modifier la simulation, respectant une séparation claire entre lecture et modification des données.
- La classe SimulationSerializer utilise un format binaire simple avec magic number, version, dimensions, topologie, nombre de pas et liste des agents.
- La version CommandLineApp permet de tester une partie du modèle sans dépendre de JavaFX.

Pour la gestion visuelle, GridView dessine les cellules sur un Canvas JavaFX en associant une couleur à chaque état. Le panneau ControlPanel rassemble tous les contrôles utilisateur, tandis que MainView relie les boutons et les actions à la grille et au moteur.

8. Résultats obtenus

Le résultat obtenu est une application fonctionnelle de simulation 2D de cellules. La version graphique permet de créer une population, de la modifier, de lancer la simulation et d'observer son évolution en direct.

Fonctionnalité	Résultat obtenu
Affichage de la grille	La grille est affichée dans un Canvas JavaFX avec une couleur différente selon l'état de chaque cellule.
Contrôle de la simulation	L'utilisateur peut lancer, mettre en pause, réinitialiser, vider la grille et exécuter pas à pas.
Interaction utilisateur	L'utilisateur peut ajouter, supprimer, peindre une zone 3x3 ou remplir un rectangle.
Paramètres dynamiques	Des sliders permettent de régler la vitesse, le taux de contagion, de récupération et de mortalité.
Topologie	La grille peut être bornée ou toroïdale, modifiant le calcul du voisinage aux bords.
Statistiques	Compteurs par état, pourcentages, âge moyen, énergie moyenne et historique graphique.
Sauvegarde/chargement	L'état courant peut être enregistré dans un fichier .sim puis restauré.
Test console	La version console produit des statistiques sur une grille d'exemple.

Sur le plan pédagogique, le projet montre une bonne utilisation de la programmation orientée objet : les objets possèdent des responsabilités identifiables, les règles métier sont séparées de l'affichage, et l'utilisateur peut agir directement sur le modèle via l'interface.

9. Limites du projet

Même si le projet atteint ses objectifs principaux, plusieurs limites peuvent être relevées :

- Le projet ne possède pas de fichier Maven ou Gradle, ce qui rend la compilation manuelle plus contraignante.
- Le chemin du SDK JavaFX doit être adapté à chaque ordinateur, ce qui peut gêner l'exécution.
- Les règles utilisent `Math.random()`, donc les simulations ne sont pas reproductibles exactement.
- Il n'existe pas de suite complète de tests unitaires automatisés pour vérifier toutes les règles.
- La sauvegarde impose de charger le fichier dans une grille de mêmes dimensions, ce qui limite la flexibilité.
- Les classes principales ne sont pas toutes rangées dans des packages Java structurés.
- L'interface n'offre pas encore d'export des résultats ni de courbes détaillées sur une longue période.
- Le modèle biologique reste volontairement simplifié et ne cherche pas à représenter un modèle scientifique précis.

10. Améliorations possibles

Plusieurs améliorations peuvent être envisagées pour enrichir le projet et le rendre plus professionnel :

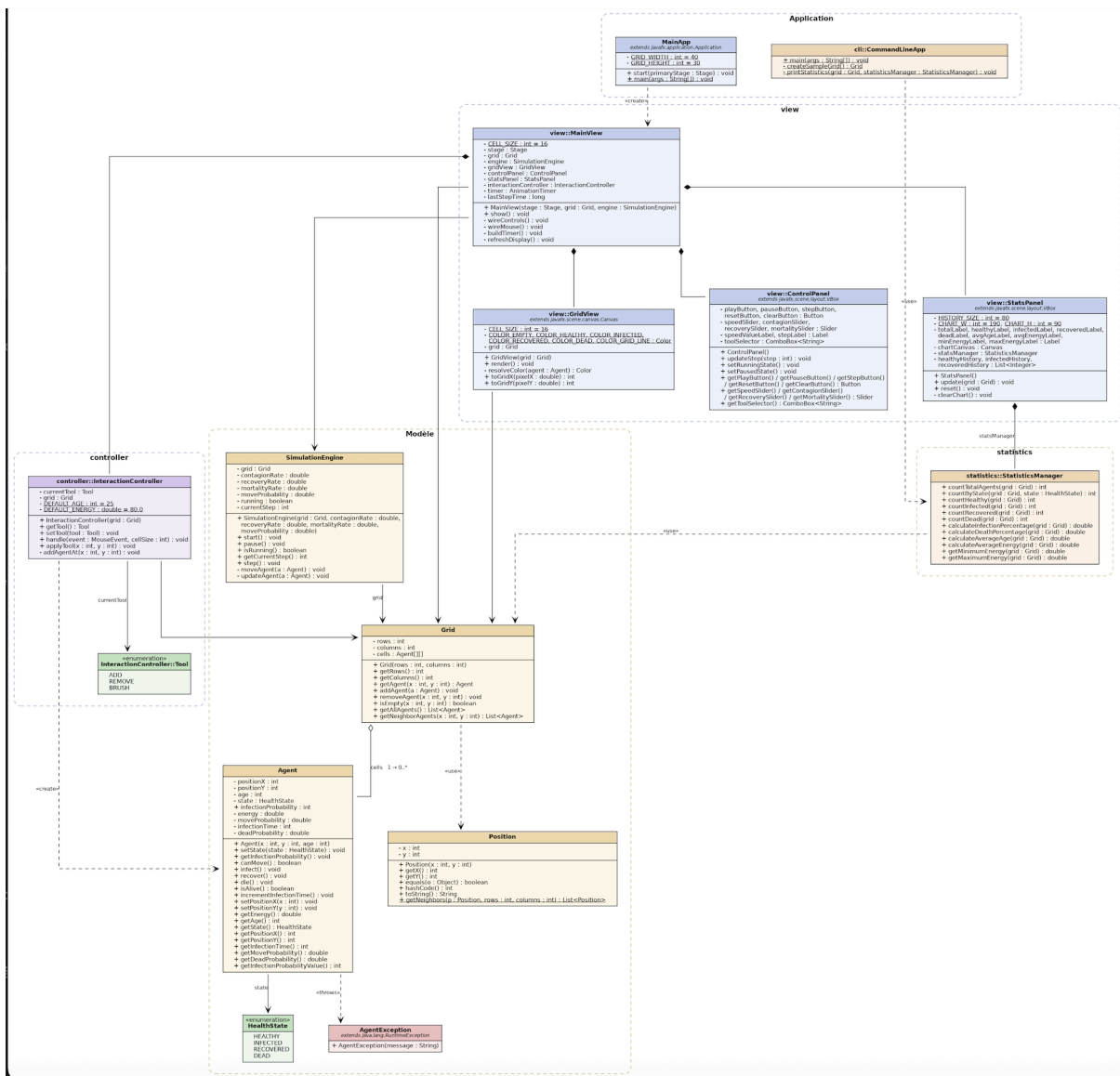
Amélioration	Intérêt
Créer un projet Maven ou Gradle	Automatiser la compilation, la gestion de JavaFX et l'exécution sur plusieurs machines.
Ajouter des tests unitaires JUnit	Vérifier les règles d'infection, de récupération, de mort, de division, de voisinage et de sauvegarde.
Graine aléatoire configurable	Permettre de rejouer exactement la même simulation pour comparer plusieurs paramètres.
Améliorer le modèle de simulation	Ajouter des types de cellules, des obstacles, des ressources, des immunités ou des probabilités plus réalistes.
Exporter les statistiques	Permettre la sauvegarde des résultats en CSV ou l'affichage de graphiques plus détaillés.
Améliorer l'interface	Ajouter une légende, un mode sombre/clair, un zoom, une prévisualisation de zone ou des messages d'aide.
Structurer les packages	Séparer clairement model, view, controller, service, persistence et statistics.
Taille de grille configurable	Permettre à l'utilisateur de choisir la largeur, la hauteur et la taille des cellules.

L'amélioration la plus importante serait de transformer le projet en application structurée avec packages, gestionnaire de dépendances et tests automatisés. Cela rendrait le code plus simple à maintenir, plus facile à partager et plus fiable lors des évolutions futures.

11. Diagramme UML

Le diagramme UML ci-dessous représente les principales classes du projet et leurs relations. Il est volontairement simplifié pour rester lisible : il montre les classes centrales, leurs responsabilités et les dépendances principales.

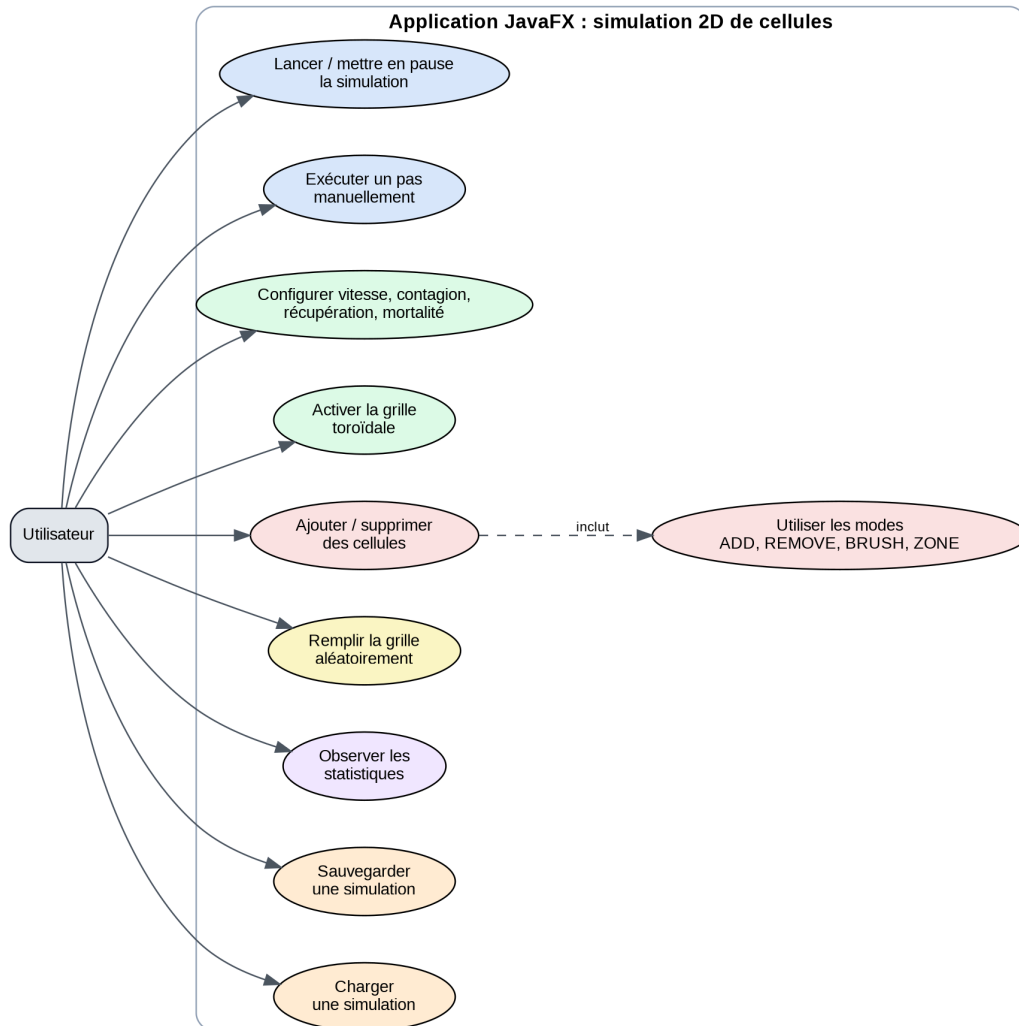
Figure 1 — Diagramme UML simplifié des principales classes du projet.



12. Cas d'utilisation

Les cas d'utilisation décrivent les interactions principales entre l'utilisateur et l'application. L'acteur principal est l'utilisateur qui manipule la simulation depuis l'interface JavaFX.

Figure 2 — Diagramme des cas d'utilisation principaux.



Le scénario nominal d'utilisation est le suivant : l'utilisateur lance l'application, place ou génère des cellules, configure les paramètres, démarre la simulation, observe l'évolution, met en pause si nécessaire, puis sauvegarde l'état obtenu. Il peut ensuite recharger cette simulation pour poursuivre l'observation à partir du même point.

Conclusion

Ce projet a permis de réaliser une application complète autour d'une simulation de cellules 2D. Il mobilise plusieurs notions essentielles de génie logiciel : modélisation orientée objet, séparation des responsabilités, interface graphique, gestion d'événements, calcul de statistiques et sauvegarde de données.

Malgré certaines limites techniques, notamment l'absence de gestionnaire de dépendances et de tests automatisés complets, le projet fournit une base solide et évolutive. La structure actuelle permet déjà

d'expliquer clairement le fonctionnement de l'application et de proposer des évolutions réalistes. Les prochaines améliorations devraient viser la robustesse, la reproductibilité, l'organisation en packages et l'enrichissement des fonctionnalités statistiques.